

LAPORAN FINAL PROJECT STRUKDAT SISTEM PUBLIC TRAIN TRANSPORT PLANNER MENGUNAKAN GRAPH DAN TREE

Kelompok 2 :

1. Hasheemi Rafsanjani - 5027251015
2. Hendra Manudinata - 5027251051
3. Nabila Sharliz Sigit - 5027251054
4. Rido Patra Yudhistira Edwin - 5027251120
5. Rayhan Fadhilah Allayn - 5027151126

I. Deskripsi Masalah

Seiring meningkatnya penggunaan transportasi umum terutama perkeretaapian, pengguna sering mengalami kesulitan dalam menentukan rute perjalanan yang paling sesuai dengan kebutuhan mereka. Meskipun informasi rute kereta sudah tersedia melalui berbagai platform, pengguna masih harus membandingkan sendiri berbagai alternatif perjalanan untuk mendapatkan rute yang efisien. Selain itu, suatu rute belum tentu optimal dari segi waktu tempuh, jarak perjalanan, maupun biaya yang dikeluarkan. Oleh karena itu, diperlukan sebuah sistem Public Train Transport Planner yang dapat membantu pengguna menemukan rute terbaik berdasarkan kriteria tertentu sehingga proses perencanaan perjalanan menjadi lebih mudah dan efisien.

II. Dataset

Dataset dibagi menjadi 2 berdasarkan daerah, jatim dan non-jatim yang masing-masing memiliki dataset routes (edges) dan stasiunnya (vertex). Berikut adalah link spreadsheet dari Datasetnya.

[Dataset Public Train Transport Planner](#)

Dataset ini mempunyai skenario pencarian stasiun, pencarian rute tercepat, pencarian rute minim transit, visualisasi struktur graph serta edge cases seperti menangani rute yang terisolasi atau struktur graph nya terputus, penghapusan rute tidak akan menyebabkan rute lain menggantung karena putus, dan menghindari nama stasiun ganda.

III. Struktur Graph yang digunakan

Struktur graph yang digunakan adalah Adjacency List karena efisien dalam menyimpan hubungan antar stasiun, baik yang terhubung maupun yang tidak terhubung. Selain lebih hemat penggunaan memori, struktur ini juga mendukung penelusuran stasiun tetangga (neighbor traversal) yang cepat serta mudah dikembangkan ketika terdapat penambahan stasiun atau rute baru. Penggunaan algoritma Breadth-First Search (BFS) seperti pada proyek ini juga seringkali menggunakan Adjacency List dikarenakan efisiensi dan kompleksitas waktu nya yang relatif ringan.

IV. Struktur Tree yang digunakan

Struktur tree yang digunakan adalah Trie karena mampu melakukan pencarian nama stasiun secara cepat berdasarkan awalan kata (prefix). Trie memiliki waktu pencarian yang konsisten sesuai panjang kata yang dicari, sehingga tetap efisien meskipun jumlah data stasiun bertambah. Selain itu, Trie mendukung fitur autocomplete dan memanfaatkan penyimpanan yang lebih efisien dengan berbagi prefix yang sama antar nama stasiun.

V. Algoritma yang digunakan

Algoritma yang digunakan yaitu ada 2, BFS dan Dijkstra. BFS digunakan untuk menelusuri graph secara menyeluruh dan mencari jalur berdasarkan jumlah pemberhentian (stops) paling sedikit. Algoritma ini mengunjungi simpul secara bertahap dari level terdekat ke level yang lebih jauh. Dijkstra digunakan untuk mencari rute optimal dengan bobot terkecil, seperti waktu tempuh, jarak, atau biaya perjalanan. Algoritma ini memastikan rute yang ditemukan merupakan rute terbaik berdasarkan kriteria yang dipilih pengguna.

VI. Design Decision Log

No	Keputusan	Alternatif	Alasan Memilih	Risiko/Kelemahan
1	Menggunakan adjacency list (Graph.java)	Adjacency matrix	Dataset stasiun kita termasuk <i>sparse graph</i> (25 node, 42 rute). Jauh lebih hemat memori ($O(V + E)$) dibanding matriks ($O(V^2)$).	Proses pengecekan rute langsung antara dua stasiun spesifik harus traverse list sehingga

				lebih lambat ($O(\text{degree}(V))$).
2	Menggunakan Trie (<code>Trie.java</code>)	HashMap atau Array biasa	Mengakomodasi fitur pencarian stasiun berbasis awalan kata (<i>prefix/autocomplete</i>) dengan kompleksitas waktu konstan $O(L)$.	Konsumsi memori lebih besar karena setiap node karakter menyiapkan array pointer berisi 128 slot ASCII.
3	Menggunakan Min-Heap manual (<code>MinHeap.java</code>)	Sorting array dinamis secara linear	Berfungsi sebagai <i>Priority Queue</i> agar proses pengambilan stasiun terdekat (<i>extract-min</i>) pada Dijkstra optimal di efisiensi $O(\log V)$.	Struktur kode menjadi jauh lebih rumit karena harus menulis logika <i>bubble-up</i> , <i>bubble-down</i> , dan peta indeks posisi sendiri.
4	Memisahkan bobot waktu dan biaya (<code>Route.java</code>)	Menggabungkan bobot pakai satu rumus utilitas	Agar fitur perbandingan (HOTS) akurat. User bisa mencari rute yang murni paling cepat atau murni paling murah tanpa bias parameter.	Perlu <i>handling state</i> (<code>MODE_TIME</code> dan <code>MODE_COST</code>) secara terpisah saat proses <i>relaxation</i> di dalam algoritma Dijkstra.

5	Menggunakan BFS murni (BFS.java)	Dijkstra dengan semua bobot diset = 1	Sifat <i>level-order traversal</i> pada BFS menjamin stasiun tujuan yang pertama kali ditemukan pasti memiliki jumlah transit (lompatan rute) paling sedikit.	BFS tidak membaca bobot menit atau biaya tiket fisik, sehingga berisiko menyarankan rute minim transit tapi memutar jauh secara geografis.
---	---	---------------------------------------	---	--

VII. Tracing Manual

1. Tracing Tree

1. Tracing Tree - Insert

Pada mulanya insert akan dilakukan pada Trie menggunakan `trie.insert(s.getName(), s.getId());` yang memecah string name menjadi per-huruf atau karakter ASCII dan memasukkannya pada struktur Tree-like. Tracing insert "Manggarai", dimulai dengan lowercase string kemudian di split per character kemudian divisualisasikan dalam blok di bawah ini :

1 [i=0] Baca 'm' (ASCII 109)

```
curr = root
root.children[109] == null → buat TrieNode('m')
curr = TrieNode('m')
```

2 [i=1] Baca 'a' (ASCII 97)

```
curr = TrieNode('m')
children[97] == null → buat TrieNode('a')
curr = TrieNode('a')
```

3 [i=2] Baca 'n' (ASCII 110)

```
curr = TrieNode('a')
children[110] == null → buat TrieNode('n')
curr = TrieNode('n')
```

4 **[i=3] Baca 'g' (ASCII 103)**
curr = TrieNode('n')
children[103] == null → buat TrieNode('g')
curr = TrieNode('g')

5 **[i=4] Baca 'g' (ASCII 103) — lagi**
curr = TrieNode('g')
children[103] == null → buat TrieNode('g') baru
curr = TrieNode('g') [depth=5]

6 **[i=5] Baca 'a'**
curr = TrieNode('g')[depth=5]
children[97] == null → buat TrieNode('a')
curr = TrieNode('a') [depth=6]

7 **[i=6] Baca 'r'**
curr = TrieNode('a')[depth=6]
children[114] == null → buat TrieNode('r')
curr = TrieNode('r')

8 **[i=7] Baca 'a'**
curr = TrieNode('r')
children[97] == null → buat TrieNode('a')
curr = TrieNode('a') [depth=8]

9 **[i=8] Baca 'i' — karakter terakhir**
curr = TrieNode('a')[depth=8]
children[105] == null → buat TrieNode('i')
curr = TrieNode('i')
Loop selesai (depth = length = 9)

10 **[DONE] Set terminal node**
curr.isEnd = true (sebelumnya false)
curr.stationId = 1
count++ → count = 1
Insert selesai: 9 node baru dibuat di Trie

Setelah muncul stationId maka insert telah sukses dilakukan dan string nama stasiun/halte sudah masuk ke data dan dapat dilakukan search

2. Tracing Tree - Search

Pada tracing ini kami menggunakan string “Mang” sebagai contoh untuk mengharapkan keluaran berupa “Manggarai” sebagai hasil autocomplete nya. Seperti pada sebelumnya string input akan di *lowercase* kemudian di *split* per character. Prosesnya yaitu pada character “M” cek pada Trie apakah ada, ada, lanjut ke “A” cek lagi kemudian seterusnya sampai akhir “G” setelah itu collect all semua data sisanya dan masukkan stationId nya ke sebuah array result list. Untuk lebih jelasnya dapat dilihat di visualisasi berikut:

1 **Lowercase input** → 'mang'
query = 'mang'.toLowerCase() = 'mang'

2 **findNode('mang') — traversal**
i=0: curr = root → root.children['m'] ada → curr = TrieNode('m')
i=1: curr.children['a'] ada → curr = TrieNode('a')
i=2: curr.children['n'] ada → curr = TrieNode('n')
i=3: curr.children['g'] ada → curr = TrieNode('g')
return TrieNode('g') — prefix node ditemukan

3 **collectAll(TrieNode('g'), results)**
DFS dari node 'g': g → g → a → r → a → i [isEnd=true]
Tambahkan stationId=1 ke results list
return results = [1]

4 **Output akhir**
searchByPrefix('Mang') → [1]
graph.getStation(1) → Station{id=1, name='Manggarai', city='Jakarta Selatan', type='KRL'}
✓ Ditemukan 1 stasiun dengan prefix 'Mang'

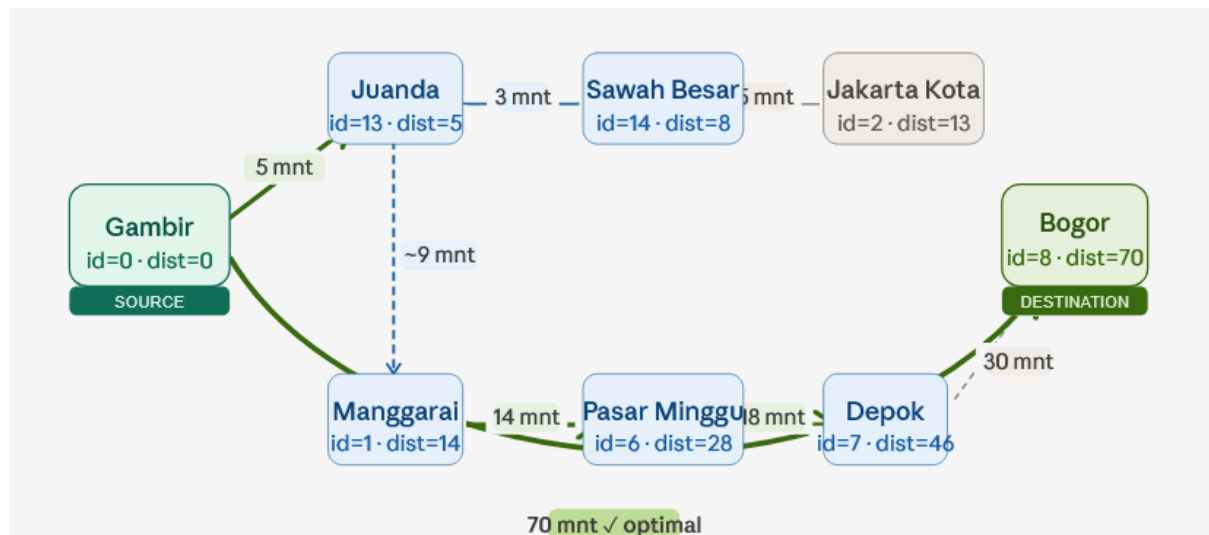
Berikut ini adalah screenshot terminal ketika dicontohkan pada string yang lain.

```
Masukkan nama atau awalan stasiun: b
✓Ditemukan 5 stasiun dengan awalan "b":
• [ 9] Bekasi          | Bekasi          | KRL    | (-6.2428, 107.0015)
• [22] Blok M         | Jakarta Selatan | MRT    | (-6.2441, 106.7975)
• [21] BNI City       | Jakarta Selatan | MRT    | (-6.2009, 106.8235)
• [ 8] Bogor          | Bogor          | KA     | (-6.5950, 106.7920)
• [24] Bundaran HI    | Jakarta Pusat  | MRT    | (-6.1944, 106.8229)
```

Untuk tracing kompleksitas pada search atau autocomplete ini adalah $O(N)$ dengan N adalah panjang string inputan searchByPrefix. Dibandingkan dengan search semua node ataupun array dan percabangan konvensional maka bisa saja terjadi kondisi terburuknya yaitu kompleksitas $O(N^2)$

2. Tracing Graph

Algoritma yang digunakan adalah Dijkstra, algoritma yang dirancang untuk mencari rute tercepat dengan edge berbobot tak negatif, penerapan pada rancangan aplikasi ini terpenuhi karena waktu tempuh dan biaya sudah disetting pada dataset untuk tidak dapat negatif. Sedangkan untuk menyimpan atau mengurutkan rute teroptimal nya digunakan min-heap sehingga dia akan memproses rute termurah/tercepat yang ditemui terlebih dahulu kemudian menyusunnya . sehingga saat menampilkan hasil langsung peek paling optimal.



Dari	Ke	Waktu (menit)	Biaya (Rp)	Jalur
0 (Gambir)	13 (Juanda)	5	2.000	Feeder Gambir-Juanda
0 (Gambir)	8 (Bogor)	70	35.000	KA Pangrango (langsung)
13 (Juanda)	14 (Sawah Besar)	3	3.000	KRL Bogor Line
14 (Sawah Besar)	2 (Jakarta Kota)	5	3.000	KRL Bogor Line
1 (Manggarai)	6 (Pasar Minggu)	14	4.000	KRL Bogor Line
6 (Pasar Minggu)	7 (Depok)	18	4.000	KRL Bogor Line
7 (Depok)	8 (Bogor)	30	4.000	KRL Bogor Line

Kami melakukan simulasi pada relasi antar stasiun bogor dan gambir , untuk mencari rute yang paling optimal dari segi waktu tempuh. Terdapat beberapa pilihan yaitu ada yang langsung ke stasiun dan ada yang transit di beberapa halte. Tracing dimulai dengan pendefinisian state awal untuk dijkstra.

Variabel	Nilai Awal
dist[]	dist[0]=0, dist[semua lain]=MAX_VALUE (∞)
parent[]	parent[semua]=-1 (belum ada parent)
visited[]	semua = false
MinHeap	insert(nodeld=0, priority=0) → Heap: [(0:0)]

Kemudian dilanjutkan dengan Proses tracing pada gambar menunjukkan penerapan algoritma Dijkstra untuk menentukan rute dengan waktu tempuh tercepat dari node asal Gambir (node 0) menuju node tujuan Bogor (node 8). Pada setiap iterasi, algoritma memilih node yang memiliki nilai waktu tempuh sementara (distance) paling kecil melalui proses extractMin(). Setelah node dipilih, seluruh tetangganya diperiksa untuk mengetahui apakah terdapat jalur yang menghasilkan waktu tempuh lebih singkat dibandingkan nilai yang telah tersimpan sebelumnya. Jika ditemukan waktu tempuh yang lebih kecil, maka nilai distance

dan parent pada node terkait akan diperbarui. Sebagai contoh, pada iterasi pertama node Gambir memperbarui waktu tempuh menuju Juanda menjadi 5 menit dan menuju Bogor menjadi 70 menit, kemudian pada iterasi berikutnya node Juanda memperbarui waktu tempuh menuju Sawah Besar dan Manggarai.

Proses pembaruan waktu tempuh terus dilakukan hingga node tujuan ditemukan. Dari hasil tracing terlihat bahwa jalur Gambir > Juanda > Sawah Besar > Manggarai > Pasar Minggu > Depok > Bogor menghasilkan total waktu tempuh sebesar 76 menit. Namun, algoritma juga menemukan bahwa terdapat rute langsung Gambir > Bogor dengan waktu tempuh 70 menit, sehingga jalur melalui beberapa stasiun tersebut tidak dipilih karena membutuhkan waktu lebih lama. Ketika node Bogor (node 8) diekstrak dari priority queue dengan nilai waktu tempuh minimum 70 menit, algoritma menghentikan proses karena rute tercepat telah ditemukan. Dengan demikian, hasil akhir menunjukkan bahwa rute optimal dari Gambir ke Bogor adalah jalur langsung Gambir > Bogor dengan total waktu tempuh 70 menit.

Berikut ini adalah visualisasi pengambilan extractMin dan lengkap dari iterasi dijkstra:

1 Iterasi 1: extractMin() → u=0 (Gambir), dist=0

Heap sebelum : [(0:0)]

visited[0] = true

Proses tetangga node 0:

→ tetangga 13 (Juanda) : $\text{dist}[0] + 5 = 5 < \text{dist}[13] = \infty \rightarrow \text{UPDATE } \text{dist}[13] = 5, \text{parent}[13] = 0$

→ tetangga 8 (Bogor) : $\text{dist}[0] + 70 = 70 < \text{dist}[8] = \infty \rightarrow \text{UPDATE } \text{dist}[8] = 70, \text{parent}[8] = 0$

heap.insert(13, 5), heap.insert(8, 70)

Heap setelah : [(13:5), (8:70)]

2 Iterasi 2: extractMin() → u=13 (Juanda), dist=5

Heap sebelum : [(13:5), (8:70)]

visited[13] = true

Proses tetangga node 13:

→ tetangga 14 (Sawah Besar) : $\text{dist}[13] + 3 = 8 < \infty \rightarrow \text{UPDATE } \text{dist}[14] = 8, \text{parent}[14] = 13$

→ tetangga 1 (via KRL link): misalkan $\text{dist} = 14 < \infty \rightarrow \text{UPDATE } \text{dist}[1] = 14, \text{parent}[1] = 13$

heap.insert(14, 8), heap.insert(1, 14)

Heap setelah : [(14:8), (1:14), (8:70)]

3 Iterasi 3: extractMin() → u=14 (Sawah Besar), dist=8

visited[14] = true

Proses tetangga: → node 2 (Jakarta Kota): $\text{dist}[14] + 5 = 13 \rightarrow \text{UPDATE } \text{dist}[2] = 13, \text{parent}[2] = 14$

Heap setelah : [(1:14), (2:13→bubbleUp), (8:70)]

4 Iterasi 4: extractMin() → u=1 (Manggarai), dist=14

visited[1] = true

Proses tetangga node 1:

→ node 6 (Pasar Minggu): $\text{dist}[1] + 14 = 28 < \infty \rightarrow \text{UPDATE } \text{dist}[6] = 28, \text{parent}[6] = 1$

→ node 11 (Cikini) : $\text{dist}[1] + 5 = 19 < \infty \rightarrow \text{UPDATE } \text{dist}[11] = 19, \text{parent}[11] = 1$

→ node 10 (Jatinegara) : $\text{dist}[1] + 8 = 22 < \infty \rightarrow \text{UPDATE } \text{dist}[10] = 22, \text{parent}[10] = 1$

Heap setelah : [(11:19), (10:22), (6:28), (8:70), ...]

5 Iterasi 5: extractMin() → u=6 (Pasar Minggu), dist=28

visited[6] = true

Proses tetangga node 6:

→ node 7 (Depok): $\text{dist}[6] + 18 = 46 < \infty \rightarrow \text{UPDATE } \text{dist}[7] = 46, \text{parent}[7] = 6$

Heap setelah : [..., (7:46), (8:70)]

6 Iterasi 6: extractMin() → u=7 (Depok), dist=46

visited[7] = true

Proses tetangga node 7:

→ node 8 (Bogor): $\text{dist}[7] + 30 = 76$ vs $\text{dist}[8] = 70 \rightarrow 76 > 70$, TIDAK UPDATE

$\text{dist}[8]$ tetap 70 (via rute langsung Gambir→Bogor KA Pangrango)

Heap setelah : [(8:70), ...]

7 Iterasi 7: extractMin() → u=8 (Bogor), dist=70 ← DESTINATION!

visited[8] = true

$u == \text{destId} (8) \rightarrow \text{BREAK}$ — algoritma berhenti

$\text{dist}[8] = 70$ menit (rute optimal ditemukan)

3. Tracing Edge Case

a. Tracing Edge Case - Same station

Untuk pengujian sebuah program, edge case adalah suatu kasus dimana data sangat atau bersinggungan dengan area dimana akan terjadi bug maupun error. Pada rancangan public transport planner ini ada beberapa case yang kami angkat yaitu dimana user tidak sengaja mencari rute dari dan ke stasiun yang sama sehingga tidak akan terjadi rute, kasus itu dapat dijelaskan melalui blok visualisasi ini

1 [Input] User memasukkan source=0, dest=0

$\text{resolveStation}(\text{'Gambir'}) \rightarrow 0$

$\text{resolveStation}(\text{'Gambir'}) \rightarrow 0$ (lagi)

2 [Guard] pickSourceDest() mendeteksi src == dest

$\text{if } (\text{src} == \text{dest}) \rightarrow \text{System.out.println}(\text{'x Asal dan tujuan sama.'})$

return null ← tidak lanjut ke algoritma

Algoritma Dijkstra/BFS TIDAK pernah dipanggil

3 [Safe] Sistem aman, tidak ada infinite loop atau hasil salah

Penanganan dilakukan di layer UI sebelum menyentuh Graph

Output: 'x Asal dan tujuan sama.'

Cara mengatasinya adalah sesimpel menambahkan if guard untuk mendeteksi kesamaan antara kedua argumen fungsi. Sehingga nantinya BFS maupun Dijkstra tidak akan dijalankan.

```
— [2] RUTE TERCEPAT —  
E (Masukkan ID angka atau nama stasiun)  
Stasiun ASAL : gam  
Stasiun TUJUAN : gam  
X Asal dan tujuan sama.
```

b. Tracing Edge Case - Node not connected

Case selanjutnya ialah dimana tidak ada edge yang ditemukan oleh algoritma. Mungkin saja ada stasiun mati atau terpencil yang benar-benar tidak memiliki hubungan dengan stasiun manapun, tapi user tidak tersengaja memilih stasiun tersebut, pada penerapan djikstra nya maka proses akan menjelajahi semua node yang terhubung hingga semua edge dengan asal stasiun awal input sudah mark terjelajahi semua, dan queue yang ada menjadi empty sehingga setelah proses iterasi selesai dilakukan percabangan apakah queue empty , jika ia maka rute dari stasiun asal ke tujuan tidak pernah terkunjungi alias tidak ada

Berikut ini adalah visualisasi dari tracing tersebut..

1 [Init] Inisialisasi BFS

```
visited[0..99] = false  
parent[0..99] = -1  
queue = SimpleQueue, enqueue(0)  
visited[0] = true
```

2 [Pop 0] dequeue() → u=0 (Gambir)

```
u=0 ≠ destId=99, lanjut  
Tetangga node 0: [13(Juanda), 8(Bogor)]  
→ enqueue(13), visited[13]=true, parent[13]=0  
→ enqueue(8), visited[8]=true, parent[8]=0  
Queue: [13, 8]
```

3 [Pop 13] dequeue() → u=13

```
Tetangga 13: [14, 1, 0(visited)]  
→ enqueue(14), enqueue(1) — keduanya belum dikunjungi  
Queue: [8, 14, 1]
```

4 [...] Proses berlanjut ke semua node yang terhubung

```
Node 8, 14, 1, 6, 7, 2, 3, 4, 5, 9, 10, 11, 12, ... dst  
Semua 25 node yang ada di jaringan akan dikunjungi  
Node 99 TIDAK ADA di adjacency list manapun → tidak pernah dienqueue  
Queue akhirnya menjadi kosong
```

5 [EMPTY] queue.isEmpty() = true — LOOP BERAKHIR

```
found = false (belum pernah mencapai u==destId==99)  
BFS selesai tanpa menemukan destination
```

```
— [2] RUTE TERCEPAT —  
(Masukkan ID angka atau nama stasiun)  
Stasiun ASAL : gam  
Stasiun TUJUAN : tas  
► Gambir → tasik  
  
🕒 Hasil Rute Tercepat:  
X Tidak ada rute yang ditemukan.
```

VIII. Screenshot Hasil Program

PUBLIC TRANSPORT PLANNER

Stasiun: 25 | Rute: 62
Struktur : Graph (Adjacency List) + MinHeap
 : Trie untuk pencarian nama
Algoritma: Dijkstra (waktu/biaya) + BFS

MENU UTAMA

1. Cari stasiun berdasarkan nama
2. Cari rute tercepat (Dijkstra-Waktu)
3. Cari rute termurah (Dijkstra-Biaya)
4. Cari rute transit minimum (BFS)
5. Bandingkan 3 kriteria rute
6. Simulasi rute tidak tersedia
7. Tampilkan semua stasiun
8. Tampilkan struktur graph
9. Manajemen data (Insert/Update/Delete)
0. Keluar

Pilihan:

ID	Nama Stasiun	Kota	Tipe
0	Gambir	Jakarta Pusat	KA
1	Manggarai	Jakarta Selatan	KRL
2	Jakarta Kota	Jakarta Utara	KRL
3	Tanah Abang	Jakarta Pusat	KRL
4	Duri	Jakarta Barat	KRL
5	Sudirman	Jakarta Selatan	KRL
6	Pasar Minggu	Jakarta Selatan	KRL
7	Depok	Depok	KRL
8	Bogor	Bogor	KA
9	Bekasi	Bekasi	KRL
10	Jatinegara	Jakarta Timur	KRL
11	Cikini	Jakarta Pusat	KRL
12	Gondangdia	Jakarta Pusat	KRL
13	Juanda	Jakarta Pusat	KRL
14	Sawah Besar	Jakarta Pusat	KRL
15	Kemayoran	Jakarta Pusat	KRL
16	Pasar Senen	Jakarta Pusat	KRL
17	Gang Sentiong	Jakarta Pusat	KRL
18	Rajawali	Jakarta Utara	KRL
19	Ancol	Jakarta Utara	KRL
20	Tanjung Priok	Jakarta Utara	KRL
21	BNI City	Jakarta Selatan	MRT
22	Blok M	Jakarta Selatan	MRT
23	Lebak Bulus	Jakarta Selatan	MRT
24	Bundaran HI	Jakarta Pusat	MRT
Total: 25 stasiun			

STRUKTUR GRAPH				
[0]	Gambir	→		
	├─ Juanda		5 mnt Rp 2,000	Feeder Gambir-Juanda
	├─ Bogor		70 mnt Rp 35,000	KA Pangrango
	├─ Bekasi		55 mnt Rp 25,000	KA Jarak Jauh
[1]	Manggarai	→		
	├─ Pasar Minggu		14 mnt Rp 4,000	KRL Bogor Line
	├─ Cikini		5 mnt Rp 3,000	KRL Bogor Line
	├─ Jatinegara		8 mnt Rp 4,000	KRL Bekasi Line
	├─ Sudirman		5 mnt Rp 3,000	KRL Loop Line
	├─ Pasar Senen		8 mnt Rp 3,000	KRL Loop Line
[2]	Jakarta Kota	→		
	├─ Sawah Besar		5 mnt Rp 3,000	KRL Bogor Line
	├─ Kenayoran		8 mnt Rp 3,500	KRL Tanjung Priok Branch
	├─ Duri		10 mnt Rp 3,500	KRL Rangkasbitung Line
[3]	Tanah Abang	→		
	├─ Sudirman		6 mnt Rp 3,000	KRL Loop Line
	├─ Duri		8 mnt Rp 3,500	KRL Rangkasbitung Line
	├─ Bundaran HI		10 mnt Rp 5,000	TransJakarta Corridor 1
	├─ Blok M		18 mnt Rp 3,500	TransJakarta Blok M-Tanah Abang
[4]	Duri	→		
	├─ Tanah Abang		8 mnt Rp 3,500	KRL Rangkasbitung Line
	├─ Jakarta Kota		10 mnt Rp 3,500	KRL Rangkasbitung Line
[5]	Sudirman	→		
	├─ Tanah Abang		6 mnt Rp 3,000	KRL Loop Line
	├─ Manggarai		5 mnt Rp 3,000	KRL Loop Line
	├─ BNI City		2 mnt Rp 0	Interchange Sudirman-BNI City
	├─ Bundaran HI		2 mnt Rp 0	Interchange BundaranHI-Sudirman
[6]	Pasar Minggu	→		
	├─ Manggarai		14 mnt Rp 4,000	KRL Bogor Line
	├─ Depok		18 mnt Rp 4,000	KRL Bogor Line
[7]	Depok	→		
	├─ Pasar Minggu		18 mnt Rp 4,000	KRL Bogor Line
	├─ Bogor		30 mnt Rp 4,000	KRL Bogor Line
[8]	Bogor	→		
	├─ Depok		30 mnt Rp 4,000	KRL Bogor Line
	├─ Gambir		70 mnt Rp 35,000	KA Pangrango
[9]	Bekasi	→		
	├─ Jatinegara		20 mnt Rp 5,000	KRL Bekasi Line
	├─ Gambir		55 mnt Rp 25,000	KA Jarak Jauh
[10]	Jatinegara	→		
	├─ Manggarai		8 mnt Rp 4,000	KRL Bekasi Line
	├─ Bekasi		20 mnt Rp 5,000	KRL Bekasi Line
[11]	Cikini	→		
	├─ Manggarai		5 mnt Rp 3,000	KRL Bogor Line
	├─ Gondangdia		4 mnt Rp 3,000	KRL Bogor Line

— [1] CARI STASIUN —————

Masukkan nama atau awalan stasiun: Depok

Ditemukan (exact match):

[7] Depok	Depok	KRL	(-6.3913, 106.8185)
------------	-------	-----	---------------------


```
Pilihan: 2

— [2] RUTE TERCEPAT —————
(Masukkan ID angka atau nama stasiun)
Stasiun ASAL : Depok
Stasiun TUJUAN : Kemayoran
► Depok → Kemayoran

Hasil Rute Tercepat:
Path : Depok → Pasar Minggu → Manggarai → Pasar Senen → Gang Sentiong → Rajawali → Kemayoran
Waktu : 55 menit
Biaya : Rp20,500
Transit: 5 kali

Pilihan: 3

— [3] RUTE TERMURAH —————
(Masukkan ID angka atau nama stasiun)
Stasiun ASAL : Depok
Stasiun TUJUAN : Kemayoran
► Depok → Kemayoran

Hasil Rute Termurah:
Path : Depok → Pasar Minggu → Manggarai → Pasar Senen → Gang Sentiong → Rajawali → Kemayoran
Waktu : 55 menit
Biaya : Rp20,500
Transit: 5 kali
```

IX. Analisi Kompleksitas

Analisis kompleksitas dilakukan untuk mengevaluasi efisiensi waktu eksekusi dan penggunaan memori dari struktur data serta algoritma yang digunakan pada sistem Public Transport Planner. Pengukuran dilakukan menggunakan notasi Big-O.

Pada analisis ini digunakan notasi sebagai berikut:

- V = jumlah stasiun (vertex)
- E = jumlah rute (edge)
- L = panjang karakter nama stasiun
- R = jumlah hasil pencarian prefix
- C = total karakter seluruh nama stasiun

Pada implementasi saat ini, sistem menggunakan 25 stasiun dan 62 rute, namun analisis dilakukan secara umum agar tetap relevan ketika jumlah data bertambah.

1. Kompleksitas Struktur Graph (Adjacency List)

Graph direpresentasikan menggunakan adjacency list yang terdiri atas array stasiun dan daftar tetangga untuk setiap stasiun.

Representasi ini dipilih karena jaringan transportasi termasuk sparse graph, yaitu jumlah rute jauh lebih kecil dibandingkan jumlah maksimum hubungan antarstasiun.

Kompleksitas operasi pada graph ditunjukkan pada Dibawah:

Operasi	Kompleksitas Waktu
Menambah stasiun	$O(1)$

Mengambil data stasiun	$O(1)$
Menambah rute	$O(1)$
Mengambil daftar tetangga	$O(1)$
Mengubah data rute	$O(\deg(v))$
Menghapus rute	$O(\deg(v))$
Menghapus stasiun	$O(V + E)$

dengan $\deg(v)$ merupakan jumlah tetangga dari suatu stasiun.

Kompleksitas ruang adjacency list adalah:
 $O(V + E)$

karena hanya menyimpan simpul dan sisi yang benar-benar digunakan.

2. Kompleksitas Struktur Trie

Trie digunakan untuk mendukung pencarian nama stasiun dan fitur autocomplete.

a. Insert

Proses insert menelusuri setiap karakter pada nama stasiun sebanyak satu kali.
 $O(L)$

b. Exact Search

Pencarian nama stasiun dilakukan dengan mengikuti jalur karakter hingga akhir kata.
 $O(L)$

c. Prefix Search

Pencarian autocomplete terdiri dari dua tahap, yaitu:

1. Menemukan node akhir prefix.
 $O(L)$
2. Mengumpulkan seluruh hasil yang sesuai.
 $O(R)$

Sehingga kompleksitas totalnya adalah:
 $O(L + R)$

d. Delete

Penghapusan nama stasiun dari Trie dilakukan dengan menelusuri setiap karakter.
 $O(L)$

Kompleksitas ruang Trie adalah:
 $O(C)$

dengan C merupakan total karakter seluruh nama stasiun yang tersimpan.

3. Kompleksitas Min-Heap

Min-Heap digunakan sebagai priority queue pada algoritma Dijkstra.

Operasi	Kompleksitas
Insert	$O(\log V)$
Extract Min	$O(\log V)$
Decrease Key	$O(\log V)$
Contains	$O(1)$

Kompleksitas ruang Min-Heap adalah:
 $O(V)$

karena jumlah elemen maksimum sama dengan jumlah stasiun.

4. Kompleksitas Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk mencari:

- rute tercepat berdasarkan waktu tempuh;
- rute termurah berdasarkan biaya perjalanan.

Implementasi Dijkstra memanfaatkan adjacency list dan Min-Heap.

Pada algoritma ini:

- setiap stasiun diproses paling banyak satu kali;
- setiap rute direlaksasi satu kali;
- operasi priority queue membutuhkan waktu $O(\log V)$.

Dengan demikian, kompleksitas waktunya adalah:
 $O((V + E) \log V)$

Pada graph yang bersifat sparse, kompleksitas tersebut dapat disederhanakan menjadi:

$O(E \log V)$

Kompleksitas ruang Dijkstra adalah:
 $O(V)$

yang berasal dari penggunaan array dist, parent, visited, dan Min-Heap.

5. Kompleksitas Algoritma Breadth-First Search (BFS)

BFS digunakan untuk mencari rute dengan jumlah transit paling sedikit.

Algoritma BFS menjelajahi graph berdasarkan level sehingga pertama kali stasiun tujuan ditemukan, jumlah transit yang diperoleh dipastikan minimum.

Setiap stasiun dikunjungi maksimal satu kali dan setiap rute diperiksa maksimal satu kali.

Kompleksitas waktu BFS adalah:

$$O(V + E)$$

Sedangkan kompleksitas ruangnya adalah:

$$O(V)$$

yang berasal dari queue, array visited, dan array parent.

6. Kompleksitas Fitur Sistem

Berikut ringkasan kompleksitas fitur utama pada aplikasi.

Fitur	Algoritma	Kompleksitas Waktu
Cari stasiun	Trie	$O(L)$
Autocomplete stasiun	Trie	$O(L + R)$
Cari rute tercepat	Dijkstra	$O((V + E) \log V)$
Cari rute termurah	Dijkstra	$O((V + E) \log V)$
Cari transit minimum	BFS	$O(V + E)$
Bandingkan tiga kriteria rute	$2 \times$ Dijkstra + BFS	$O((V + E) \log V)$
Simulasi rute baru	Dijkstra + BFS	$O((V + E) \log V)$

Fitur perbandingan tiga kriteria menjalankan algoritma Dijkstra sebanyak dua kali dan BFS sebanyak satu kali. Secara matematis dapat dituliskan sebagai:
 $2 \times O((V + E) \log V) + O(V + E)$

yang disederhanakan menjadi:

$$O((V + E) \log V)$$

7. Kesimpulan Analisis Kompleksitas

Berdasarkan hasil analisis, penggunaan adjacency list, Trie, dan Min-Heap memberikan efisiensi yang baik untuk sistem transportasi publik.

Adjacency list memungkinkan penyimpanan graph secara hemat memori dengan kompleksitas ruang $O(V + E)$. Trie mampu menyediakan fitur pencarian dan autocomplete secara cepat dengan kompleksitas $O(L)$. Sementara itu, kombinasi Dijkstra dan Min-Heap menghasilkan pencarian rute optimal dengan kompleksitas $O((V + E) \log V)$.

Secara keseluruhan, sistem memiliki kompleksitas ruang sebesar:

$$O(V + E + C)$$

dengan kompleksitas waktu terbesar berasal dari algoritma Dijkstra, yaitu:
 $O((V + E) \log V)$

Kompleksitas tersebut menunjukkan bahwa sistem masih dapat menangani penambahan jumlah stasiun dan rute dengan baik.

X. What-if Analysis

1. Apa yang terjadi jika jumlah node naik dari 25 menjadi 10.000?
 - **Struktur Data:** Penyimpanan graf (**Adjacency List**) tidak ada masalah karena skalanya linear. Namun, memori untuk **Trie** akan membengkak drastis karena banyaknya alokasi array pointer kosong dari node-node baru.
 - **Algoritma:** Kecepatan Dijkstra dengan Min-Heap akan turun secara logaritmik mengikuti $O((V+E) \log V)$. Proses pergeseran elemen di dalam Heap naik menjadi sekitar ≈ 14 kali per operasi. Aplikasi akan terasa sedikit ada latensi (dalam milidetik) tapi tidak sampai freeze.
2. Apa yang terjadi jika edge tertentu dihapus?
 - **Mekanisme Jaringan:** Jika rute transit vital (misal: "Sudirman – BNI City") dihapus lewat fitur simulasi rute tidak tersedia, relasi edge tersebut akan di-remove dari list tetangga di objek graf.
 - **Solusi Program:** Program tidak akan *crash*. Algoritma Dijkstra dan BFS otomatis mencari rute memutar alternatif lain (misal dialihkan lewat Manggarai atau bus TransJakarta). Jika stasiun tujuan benar-benar terisolasi tanpa jalur alternatif, program mengembalikan status **found = false** dan memunculkan pesan validasi "*Tidak ada rute yang ditemukan*".
3. Apa yang terjadi jika bobot edge berubah?
 - **Dampak Logika:** Kondisi ini akan memicu hasil rute yang berbeda kontras pada fitur perbandingan (HOTS). Jika waktu tempuh TransJakarta macet total hingga 90 menit, fungsi **Cari Rute Tercepat** otomatis mendeteksi bobot besar itu dan langsung mengalihkan rute user ke MRT/KRL yang bebas macet. Sebaliknya, fungsi **Cari Rute Termurah** tetap akan menyarankan TransJakarta jika harga tiketnya tidak ikut naik (tetap Rp3.500).
4. Apa yang terjadi jika input user tidak ditemukan?
 - **Ketahanan Trie:** Jika user salah ketik nama stasiun (misal: "Gambirr"), fungsi pencarian di **Trie.java** akan langsung menemui pointer **null** di tengah jalan dan berhenti secara aman dengan mengembalikan nilai ID **-1**. Hal ini menghindarkan aplikasi dari error NullPointerException dan memicu tampilan teks "*Stasiun tidak ditemukan*" pada konsol.
5. Apa yang terjadi jika graph tidak terhubung?
 - **Dampak pada Dijkstra:** Jika ada stasiun baru yang terisolasi tanpa rute tunggal pun yang terhubung, array jarak (**dist[]**) untuk stasiun tersebut akan selamanya bernilai tak hingga (**Integer.MAX_VALUE**). Antrean Min-Heap akan terus berjalan sampai habis dieksplorasi, lalu logika program akan langsung mendeteksi bahwa stasiun tujuan tidak dapat dijangkau dari lokasi asal.

XI. Kesimpulan

Berdasarkan hasil perancangan dan implementasi, dapat disimpulkan bahwa struktur data dan algoritma memiliki peran penting dalam menyelesaikan permasalahan nyata, khususnya pada perencanaan perjalanan transportasi publik.

Pada proyek Public Transport Planner, jaringan transportasi dimodelkan menggunakan struktur data graph dengan representasi adjacency list, di mana setiap stasiun direpresentasikan sebagai vertex dan setiap rute direpresentasikan sebagai edge yang memiliki atribut bobot berupa waktu tempuh dan biaya perjalanan.

Untuk mendukung pencarian nama stasiun secara cepat, digunakan struktur data Trie yang memungkinkan proses pencarian dan autocomplete berdasarkan awalan kata secara efisien. Sementara itu, algoritma Dijkstra yang dikombinasikan dengan Min-Heap digunakan untuk menentukan rute tercepat dan termurah, sedangkan algoritma Breadth-First Search (BFS) digunakan untuk mencari rute dengan jumlah transit paling sedikit.

Hasil pengujian menunjukkan bahwa sistem mampu menangani berbagai skenario, termasuk pencarian rute optimal berdasarkan beberapa kriteria, simulasi penambahan dan penghapusan rute, serta penanganan edge case seperti stasiun yang tidak terhubung, input stasiun yang tidak valid, dan kondisi asal serta tujuan yang sama.

Berdasarkan analisis kompleksitas, penggunaan adjacency list menghasilkan kebutuhan memori sebesar $O(V + E)$, sedangkan pencarian rute menggunakan algoritma Dijkstra memiliki kompleksitas waktu $O((V + E) \log V)$ dan BFS memiliki kompleksitas $O(V + E)$. Hasil tersebut menunjukkan bahwa sistem tetap efisien dan dapat dikembangkan untuk menangani jumlah stasiun dan rute yang lebih besar.

Dengan demikian, proyek ini membuktikan bahwa penerapan struktur data graph, Trie, Min-Heap, serta algoritma Dijkstra dan BFS mampu memberikan solusi yang efektif, efisien, dan relevan untuk permasalahan perencanaan rute transportasi publik.